Exp No: 10

Image Generation using a Generative Adversarial Network (GAN)

AIM:

To train a Generative Adversarial Network (GAN) to generate new images from the LSUN dataset, and to evaluate the quality of the generated images using visual inspection and quantitative metrics such as the Inception Score (IS) and the Fréchet Inception Distance (FID).

ALGORITHM / PROCEDURE:

1. Import PyTorch and torchvision, and load the LSUN dataset (bedroom class, a subset of about 30,000 images).
2. Resize and crop every image to 64 x 64 and normalise the pixel values to the range [-1, 1].
3. Build the Generator: a 100-dimensional noise vector is up-sampled by five transposed-convolution layers to a 64 x 64 RGB image (Tanh output).
4. Build the Discriminator: five convolution layers that classify a 64 x 64 image as real or fake (Sigmoid output).
5. Train both networks with the Adam optimiser (lr 0.0002, beta1 0.5) and Binary Cross-Entropy loss: first update the discriminator on real and fake images, then update the generator to fool the discriminator.
6. Generate images from a fixed noise batch after training and inspect them visually for realism and diversity.
7. Compute the FID (lower is better) and Inception Score (higher is better) using a pre-trained Inception network and record the values.

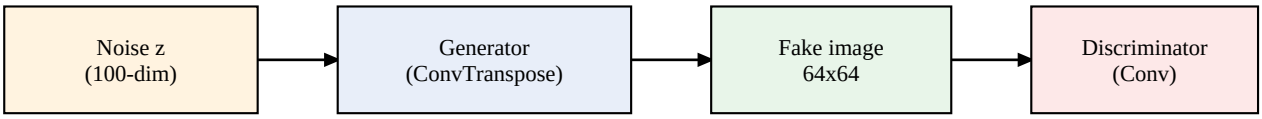

Fig 1: Architecture of the GAN (real images from LSUN are also fed to the Discriminator)

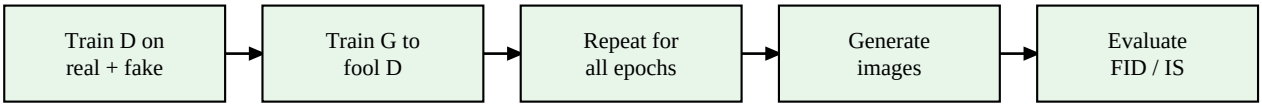

Fig 2: Working of GAN training and evaluation

Parameters used:

Parameter	Value
Dataset	LSUN (bedroom), 30,000-image subset
Image size / noise size	64 x 64 / 100
Batch size / epochs	128 / 20
Optimiser	Adam (lr 0.0002, beta1 = 0.5) for both networks
Loss function	Binary Cross-Entropy
Evaluation	Visual inspection, FID (lower is better), Inception Score (higher is better)

PROGRAM:

```
import torch, torch.nn as nn, torchvision.utils as vutils
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt
# pip install torchmetrics torch-fidelity
from torchmetrics.image.fid import FrechetInceptionDistance
from torchmetrics.image.inception import InceptionScore

device = "cuda" if torch.cuda.is_available() else "cpu"
IMG, NZ, NGF, NDF, BATCH, EPOCHS = 64, 100, 64, 64, 128, 20

# 1. Load LSUN (use one class, e.g. bedroom, and a subset of about 30,000 images)
tf = transforms.Compose([transforms.Resize(IMG), transforms.CenterCrop(IMG), transforms.ToTensor(),
                         transforms.Normalize((.5, .5, .5), (.5, .5, .5))])
ds = datasets.LSUN(root="./lsun", classes=["bedroom_val"], transform=tf)   # or ImageFolder on a Kaggle subset
ds = Subset(ds, range(min(30000, len(ds))))
loader = DataLoader(ds, batch_size=BATCH, shuffle=True, num_workers=2, drop_last=True)

# 2. DCGAN generator and discriminator
class G(nn.Module):
    def __init__(s):
        super().__init__()
        s.net = nn.Sequential(
            nn.ConvTranspose2d(NZ, NGF*8, 4, 1, 0, bias=False), nn.BatchNorm2d(NGF*8), nn.ReLU(True),
            nn.ConvTranspose2d(NGF*8, NGF*4, 4, 2, 1, bias=False), nn.BatchNorm2d(NGF*4), nn.ReLU(True),
            nn.ConvTranspose2d(NGF*4, NGF*2, 4, 2, 1, bias=False), nn.BatchNorm2d(NGF*2), nn.ReLU(True),
            nn.ConvTranspose2d(NGF*2, NGF, 4, 2, 1, bias=False), nn.BatchNorm2d(NGF), nn.ReLU(True),
            nn.ConvTranspose2d(NGF, 3, 4, 2, 1, bias=False), nn.Tanh())
    def forward(s, z): return s.net(z)

class D(nn.Module):
    def __init__(s):
        super().__init__()
        s.net = nn.Sequential(
            nn.Conv2d(3, NDF, 4, 2, 1, bias=False), nn.LeakyReLU(0.2, True),
            nn.Conv2d(NDF, NDF*2, 4, 2, 1, bias=False), nn.BatchNorm2d(NDF*2), nn.LeakyReLU(0.2, True),
            nn.Conv2d(NDF*2, NDF*4, 4, 2, 1, bias=False), nn.BatchNorm2d(NDF*4), nn.LeakyReLU(0.2, True),
            nn.Conv2d(NDF*4, NDF*8, 4, 2, 1, bias=False), nn.BatchNorm2d(NDF*8), nn.LeakyReLU(0.2, True),
            nn.Conv2d(NDF*8, 1, 4, 1, 0, bias=False), nn.Sigmoid())
    def forward(s, x): return s.net(x).view(-1)

def init_w(m):
    if isinstance(m, (nn.Conv2d, nn.ConvTranspose2d)): nn.init.normal_(m.weight, 0, 0.02)
    elif isinstance(m, nn.BatchNorm2d): nn.init.normal_(m.weight, 1, 0.02); nn.init.zeros_(m.bias)

netG, netD = G().to(device), D().to(device); netG.apply(init_w); netD.apply(init_w)
bce = nn.BCELoss()
oG = torch.optim.Adam(netG.parameters(), lr=2e-4, betas=(0.5, 0.999))
oD = torch.optim.Adam(netD.parameters(), lr=2e-4, betas=(0.5, 0.999))
fixed = torch.randn(64, NZ, 1, 1, device=device)

# 3. Training loop
for ep in range(1, EPOCHS + 1):
    for real, _ in loader:
        real = real.to(device); b = real.size(0)
        z = torch.randn(b, NZ, 1, 1, device=device); fake = netG(z)
        # train discriminator
        oD.zero_grad()
        lD = bce(netD(real), torch.ones(b, device=device)) + bce(netD(fake.detach()), torch.zeros(b, device=device))
        lD.backward(); oD.step()
        # train generator
        oG.zero_grad(); lG = bce(netD(fake), torch.ones(b, device=device)); lG.backward(); oG.step()
    print(f"Epoch [{ep}/{EPOCHS}] | D loss {lD.item():.4f} | G loss {lG.item():.4f}")

# 4. Visual inspection
with torch.no_grad(): samples = netG(fixed).cpu()
grid = vutils.make_grid(samples, padding=2, normalize=True)
plt.figure(figsize=(8, 8)); plt.axis("off"); plt.imshow(grid.permute(1, 2, 0)); plt.savefig("generated.png", dpi=150

# 5. Quantitative evaluation: FID and Inception Score
def to_uint8(x): return ((x * 0.5 + 0.5).clamp(0, 1) * 255).to(torch.uint8)
fid = FrechetInceptionDistance(feature=2048).to(device); IS = InceptionScore().to(device)
N = 0
for real, _ in loader:
```

```
    fid.update(to_uint8(real.to(device)), real=True); N += real.size(0)
    if N >= 5000: break
with torch.no_grad():
    for _ in range(N // BATCH):
        f = to_uint8(netG(torch.randn(BATCH, NZ, 1, 1, device=device)))
        fid.update(f, real=False); IS.update(f)
is_mean, is_std = IS.compute()
print(f"FID score       : {fid.compute().item():.2f}  (lower is better)")
print(f"Inception Score : {is_mean.item():.2f} +/- {is_std.item():.2f}  (higher is better)")
```

OUTPUT:

Note: the output boxes below show the exact format printed by the program. Replace the <...> values with the numbers from your own run (paste the console text or a screenshot).

```
Epoch [1/20]  | D loss <value> | G loss <value>
Epoch [2/20]  | D loss <value> | G loss <value>
...
Epoch [20/20] | D loss <value> | G loss <value>

FID score       : <value>  (lower is better)
Inception Score : <value> +/- <value>  (higher is better)

[Paste the saved image grid generated.png here for the visual inspection]
```

RESULT:

Thus a Generative Adversarial Network was trained on the LSUN dataset to generate new 64 x 64 images. The quality of the generated images was evaluated by visual inspection of the image grid (realism, sharpness and diversity) and quantitatively using the Fréchet Inception Distance and the Inception Score. A lower FID and a higher Inception Score indicate generated images that are closer to the real LSUN images.